

CeTTa:

A Direct C Runtime for MeTTa with Verified Translation

Zar Goertzel
Oruži

June 21, 2026

Abstract

MeTTa is a reflective, non-deterministic meta-language with three actively maintained runtimes: Hyperon Experimental (HE, Rust), PeTTa (Prolog), and CeTTa (C). This paper describes CeTTa, a direct C implementation of the HE MeTTa evaluator that passes a 375-file regression suite with zero failures (equally green in the MORK-bridge build), backed by a 333-case rule-indexed conformance catalog against upstream HE v0.2.10 covering 24/24 specification rule capacities, and achieves **9.5× speedup over PeTTa** on genuine depth-10 PLN backward chaining with truth-value propagation (19,845 hypotheses over 1.4M atoms in 15 s vs. PeTTa’s 143 s), while losing 13× on pure branching search (8-queens) and 30× on BFS state exploration (tilepuzzle). The runtime is organized as an explicit profile family: **he-compat** mirrors Rust HE literally (including its quirks); the principled **he** line carries exact rationals, true division, and per-case recorded divergences; **he-extended** adds labeled extensions; **he-prime** adds dependent telescopes. The one-step reduction relation is reflected into MeTTa itself as a modal surface (**transitions**, **box**, **diamond**, normal-form tests) aligned with the OSLF LanguageDef $\rightarrow$ ReductionSpan $\rightarrow$ $\Diamond/\Box$ pipeline proven in Lean, and the first rule classes (grounded dispatch, argument congruence, equation matching) are welded to a reflectable small-step rule table whose rule-removal tests prove the engine genuinely depends on the declared rules. The runtime features SymbolId-based dispatch (38% instruction reduction), structured space kinds (queue, hash, stack), a pluggable match backend with PathMap/MORK integration, and compiled space artifacts via MORK’s **.act** format for zero-copy indexed loading. It is backed by a sorry-free Lean proof (5,400+ lines) establishing semantic correspondence between HE and PeTTa evaluation on the pure fragment, including import commutativity and operational bridges for state and space allocation. A kernel-checked HE’ telescope formalization extends the type system with dependent binder semantics. The RhoCalc reaction machine’s eval-at-COMM semantics is given a witness-free, axiom-clean Lean characterization: the quiescent run of the canonical payload-evaluation process equals its structurally observable outcome set. We present a complete benchmark ladder from shallow evaluation through depth-10 PLN inference, callgrind/massif-based performance analysis, a specification gap analysis, and a three-machine architecture (local search, indexed shared-space, type elaboration) informed by GPT-5.4 Pro architectural review.

1 Introduction

MeTTa [1] is a reflective, typed, non-deterministic language designed as the core language of the OpenCog Hyperon AGI framework. Three implementations exist:

HE Hyperon Experimental (Rust, v0.2.10). The reference implementation with Python bindings and a large standard library.

PeTTa Prolog-based MeTTa (SWI-Prolog). Leverages Prolog’s SLD resolution for backward chaining and pattern matching.

CeTTa Direct C evaluator (this work). Targets the HE spec with arena allocation, discrimination-trie indexing, and tail-call optimization.

Contributions.

1. A direct C runtime passing a 375-file regression suite (0 failures) plus 104 profile tests, validated against upstream HE v0.2.10 as oracle.
2. An explicit profile family (**he-compat** / **he** / **he-extended** / **he-prime**) with per-case recorded divergences and a fail-by-default conformance harness: a 333-case rule-indexed catalog (all agree, 24/24 spec-rule capacities) and a 378-example broad corpus with every difference classified (Section 3).
3. A reflective modal LTS surface over the evaluator’s one-step reduction relation—successor set, $\Box/\Diamond$, normal-form tests—with runtime De Morgan self-tests mirroring the Galois connection proven in the Lean OSLF framework.
4. The first specification weld: a reflectable small-step rule table gating grounded dispatch, argument congruence, and equation matching, with anti-decorative rule-removal tests. This converts a cross-branch one-step soundness bug (missing argument congruence) into a structurally prevented class.
5. A bidirectional HE $\leftrightarrow$ PeTTa translator (Prolog + executable Lean), with a sorry-free Lean proof of semantic correspondence.
6. Cross-runtime benchmarks showing CeTTa 10–67 $\times$ faster than HE on shallow evaluation, PeTTa 3 $\times$ faster on deep proof search, and a callgrind analysis identifying 46% of CeTTa’s runtime as optimizable string operations.
7. A characterization of the semantic boundary between HE and PeTTa: the shared fragment (85% of programs) runs identically; the divergence is in free-variable scoping across non-deterministic branches.
8. A specification gap analysis of non-determinism plus mutable state, with 8 concrete examples showing that sequential-per-iteration semantics (CeTTa’s default) is correct in 6/8 cases, while HE’s interleaved side-effect evaluation loses atoms in real code (the pattern miner).
9. **with-space-snapshot**: a CeTTa extension for branch-isolated mutation, enabling semi-naive forward chaining and speculative search without the side-effect hazards of shared mutable state.
10. A witness-free, axiom-clean Lean characterization of the RhoCalc reaction machine’s eval-at-COMM semantics: the quiescent run of the canonical payload-evaluation process equals its structural-congruence-observed outcome set, with no certification witness in the carrier (Section 5.4).

2 CeTTa Architecture

CeTTa is a direct evaluator, not a compiler or boot pipeline. It implements the six mutual functions of the HE evaluation spec (`EvalAtom`, `MettaCall`, `InterpretExpression`, `InterpretFunction`, `InterpretArgs`, `InterpretTuple`) as C functions with a shared arena allocator.

2.1 Core Design

Arena allocation. All atoms are allocated from 64KB arena blocks. No per-atom `malloc/free`. Batch deallocation at scope exit.

Symbol interning. A hash table maps symbol strings to unique pointers, enabling $O(1)$ pointer comparison for symbol equality.

Equation index. Equations (`= lhs rhs`) are indexed by head-symbol hash into 256 buckets, with a discrimination trie within each bucket for fast LHS matching.

Substitution tree. An epoch-based discrimination tree (`SubstTree`) eliminates per-candidate variable renaming during equation queries by tagging indexed variables with monotonic epochs.

Tail-call optimization. A `TAIL_REENTER` trampoline with `goto tail_call` eliminates stack growth for recursive equation dispatch.

Unlimited fuel. Default fuel is -1 (unlimited), matching the HE spec. Stack overflow is caught via `sigaltstack + SIGSEGV` handler with a clean error message.

Two-stage stdlib. The standard library is precompiled into a binary blob at build time (`cetta-stage0` $\rightarrow$ `-compile-stdlib` $\rightarrow$ blob $\rightarrow$ `cetta`), giving 6ms startup.

2.2 Test Suite

CeTTa is validated against the HE CLI (v0.2.10) as oracle. Golden regression files are oracle-validated when introduced; `make test` replays them on every build. Current status: **375 passed, 0 failed, 40 skipped** on the default build and **371 passed, 0 failed, 44 skipped** on the MORK-bridge build (all bridge sub-lanes green), plus 104 profile tests covering module imports, foreign (Python) function dispatch, numeric-tower and profile-surface splits, and extension surfaces. Two further lanes are fail-by-default: `make test-step-rules` disables individual rule-table entries and *requires* the witnesses to fail (Section 3), and the semantic conformance suite re-runs the curated oracle corpus live against upstream HE (Section 3.2).

3 Specification-Driven Semantics: Profiles, Conformance, and the Small-Step Rule Table

CeTTa is organized around an explicit principle: *the semantics is a named object, not an emergent property of the implementation*. This section describes the profile family, the oracle-conformance campaign against upstream HE, the evaluator bug classes that campaign exposed, the reflective modal surface over the one-step reduction relation, and the first welded rule pack that makes the reduction rules themselves runtime data.

3.1 The Profile Family

Profiles are distinct semantic products, implemented as additive layers—never as visibility masks over one shared universe:

he-compat A literal mirror of Rust HE v0.2.10, including its quirks: integer division for `/`, the strict three-argument `if` surface, Rust’s library inventory behavior. Compatibility means matching upstream *behavior*, not reproducing its library set; library availability is out of scope for conformance.

he The principled line. Spec-faithful where the written specification speaks; where HE’s implementation and its specification diverge, the divergence is adjudicated and recorded per-case. Carries exact arbitrary-precision rationals, true division for `/`, floor division `//`, Euclidean `math::mod`, and the clean `math::` namespace over the Rust-heritage `*-math` ops. Extensions that accept more programs (e.g. two-argument `if`) are deliberate, recorded surface choices.

he-extended **he** plus labeled CeTTa extensions: typed space kinds, `with-space-snapshot`, the `lts` reflection surface, MORK/PathMap-backed spaces.

he-prime The dependent-telescope extension (Section 10).

Two rules keep this family honest. First, unknown symbols stay inert: MeTTa’s philosophy is to leave uninterpreted what it does not know how to interpret, so no profile ever raises an “unsupported feature” error—a profile determines what is *interpreted*, never what is *legal*. Second, profile differences are explicit splits with named tests (e.g. `profile_if_compat_arity` pins that **he-compat** rejects two-argument `if` exactly as upstream does, while **he/he-extended** accept it as a recorded extension).

3.2 Oracle Conformance

For **he-compat**, upstream HE v0.2.10 is the sole oracle, and the harness is a fail-by-default gate, not a green-manufacturing report. Comparison is by *result-bag equality*—multisets of parsed results, never byte-equality of output text—so formatting differences cannot masquerade as semantic agreement, and duplicate results (which are semantically real in MeTTa) are counted.

Curated catalog. 333 cases, every one oracle-validated and in agreement, each mapped to the specification rules it exercises: 24/24 spec-rule capacities are covered, and every capacity is classified into one of three lanes—*reduction-span* (the one-step semantics the modal layer consumes), *supporting-surface* (match, unify, grounded dispatch plumbing), and *spatial-policy* (module import, inventory, space policy). The lane split keeps policy surfaces out of the reduction relation.

Broad corpus. 378 upstream examples: 302 agree, 59 classified differences (library availability, which is out of conformance scope; path-import capabilities upstream rejects; environment and rendering differences; and adjudicated upstream drift), 7 timeouts, 10 not runnable upstream. No difference is left unclassified.

Adjudicated divergences. Where CeTTa deliberately diverges from upstream, each case carries independent grounds. Three examples: upstream’s post-0.2.10 `function/NoReturn` behavior contradicts its own written specification (a deliberate loop fix that left the document stale); upstream’s `get-doc` errors on HE’s own documentation example (an arity regression); and the

user-visible result of top-level `eval` on a non-reducible term is genuinely under-determined by the specification—both engines deviate from the literal instruction-layer reading, in opposite directions.

The catalog carries an explicit `no_runtime_trace_or_certificate_surface` flag, enforced by the suite: conformance is *testing*, and the specification is what gets *proven about* in Lean. CeTTa does not emit execution traces or certificates, and a per-run check is never presented as a proof.

3.3 Evaluator Bug Classes the Campaign Exposed

Three bug classes were found and fixed, all instructive.

The Empty algebra. When overlapping equations yield a branch that reduces to `Empty`, two dual defects appeared: the surviving sibling branch was returned without being re-driven to applicative normal form (violating the spec’s applicative order), and in other shapes `Empty` itself leaked into the visible result bag (the spec is explicit that `Empty` “is not returned among other results”). The fix treats the evaluator’s outcome set as an algebra with `Empty` as its absorbing zero and re-drives surviving branches independently of their siblings. In modal terms (Section 3.4): the successor algebra must satisfy $\Diamond \perp = \perp$.

The one-step congruence bug. The reflective one-step surface reported `(+ 1 (+ 2 3))` as a *normal form*: the one-step relation lacked argument congruence (stepping a reducible argument when the head cannot fire), while the big-step evaluator had it. Because the one-step relation and the evaluator were separately hand-maintained C branch sets, the gap replicated identically across three development branches and was invisible to every test that only exercised head-reducible terms. The bug was diagnosed through the modal surface itself ($\Box \perp$ holding on a reducible term), fixed as deterministic left-to-right congruence matching the evaluator’s applicative order—and is the direct motivation for the rule-pack weld in Section 3.5.

The switch/switch-minimal conflation. Upstream HE types `switch` as `(-> %Undefined% Expression %Undefined%)`—the scrutinee is *evaluated* before matching—while `switch-minimal` is `(-> Atom Expression Atom)`, matching the raw scrutinee structurally. CeTTa routed both spellings through one structural handler, so `(switch (+ 1 2) ((3 three) (($x $y) (got $x $y))))` returned `(got 1 2)` where upstream returns `three`. The defect was exposed not by testing but by the certification campaign (Section 3.6): modeling the rules in Lean forced a precise reading of the two upstream type contracts, and an oracle probe confirmed the divergence. The fix splits the rule into `HES_Switch` (evaluated scrutinee) and `HES_SwitchMinimal` (structural), with a distinguishing witness pair pinned in the test suite.

3.4 The Reflective Modal Surface

The library surface `lts` (generic, step-parameterized) and `lts:he` (bound to the HE evaluator’s one-step relation) reflect the one-step reduction relation into MeTTa as data:

- `lts:he:transitions` emits the one-step successor set of an atom as quoted states (zero results on quiescent atoms); `lts:he:trace-2` enumerates two-step traces.
- `lts:he:is-normal-form` is exactly $\Box \perp$ (every successor satisfies falsity, i.e. no successor exists); `lts:he:is-can-step` is $\Diamond \top$.

- `lts:he:box  $\phi$` and `lts:he:diamond  $\phi$` generalize these to arbitrary unary predicates: $\Box\phi$ holds when every one-step successor satisfies ϕ (vacuously at normal forms), $\Diamond\phi$ when some successor does.

The De Morgan law $\Box\phi = \neg\Diamond\neg\phi$ is installed as a runtime self-test; it is the executable mirror of the Galois connection proven once and for all in the Mettapedia OSLF framework, where $\text{LanguageDef} \rightarrow \text{RewriteSystem} \rightarrow \text{ReductionSpan} \rightarrow (\Diamond, \Box)$ is derived generically and `langGalois` establishes the adjunction. Native behavioral types in the sense of Native Type Theory arise as predicates over exactly this reduction span; the runtime surface and the Lean derivation describe the same object at two levels.

Multiplicity discipline: `transitions` preserves the result *bag* (duplicate successors are semantically real), while the modal predicates quotient to reachability. Evaluation and conformance compare bags; $\Diamond/\Box$ use support. Result sets are never deduplicated for speed.

3.5 The Small-Step Rule Table

The congruence bug demonstrated the structural disease: reduction rules as anonymous, duplicated C branches are invisible, unremovable, and untestable-by-absence. The cure is to make the rule set a *named, reflectable object* the evaluator provably depends on.

Three granularity strata are kept distinct. *HEFine* is the Lean `mettaHE` instruction machine (59 rules over interpreter states; the proof reference). *HE small-step* is the coarse user-visible one-step relation—one observable surface rewrite, e.g. $(+ 1 (+ 2 3)) \rightarrow (+ 1 5)$ —which is what the `lts` surface exposes. *HEBigStep* is full evaluation. The table encodes the small-step relation; its relationship to the fine machine is a collapse/simulation with explicit provenance links (each coarse rule names the fine rules that justify it), never a label identity.

The runtime object, `CettaLangDefPack`, carries: language, profile, and granularity identifiers; a schema version; a source digest over the canonical rule-descriptor text; the rule table; a disabled-rule mask populated from the `CETTA_LANGDEF_DISABLED_RULES` environment variable; and a *per-rule claim level* recording the strongest evidence behind each rule (Section 3.6). Ten rule classes are live and gate the *only* implementations of their behavior in the one-step subsystem: grounded dispatch, leftmost expression congruence, equation match, `eval`, `chain`, `let`, `let*`, `case`, `switch`, and `switch-minimal` (the last two split per their distinct upstream contracts, Section 3.3). Quote/return and empty/error quiescence are structural rows: accounted in the table with written reasons, but not gateable, since disabling them would make the semantics incoherent rather than smaller. Each row carries a provenance string naming the fine-machine rules that justify it. The pack is reflected into MeTTa by `lts:he:step-rules`.

The load-bearing test is *anti-decorative*: a dedicated lane re-runs the witness suite with each live rule disabled and *requires* the witnesses to fail (the successor set must go empty; normal-form must flip)—baseline plus ten rule-removal failures confirmed. If the witnesses still passed with a rule removed, a duplicate hidden code path would exist—precisely the disease that produced the congruence bug. A disabled special form goes *inert* (its head stays uninterpreted; congruence still applies), per the principle that unknown symbols are data, never errors. The lane fails closed in both directions and runs on both the default and MORK-bridge builds.

Claim levels are deliberately *excluded* from the digest: the digest fingerprints rule *content* (names, profiles, provenance, schema), while claims are *evidence about* that content. Upgrading a rule’s certification therefore must not—and verifiably does not—move the semantic object: the digest (`f9v1a64:12eec5b655a75904` at the time of writing) stayed fixed across every claim flip. The pack is, by design, a hand-authored object kept honest by the removal lane and the digest; we

deliberately parked an earlier plan to generate specialized C from the rule source, because the weld’s value lies in the named semantic object and its fail-closed tests, and code generation is machinery the certification program does not need.

3.6 Lean Certification of the Rule Table

Each per-rule claim takes one of three values, in increasing strength: *bag-tested-adequate* (the rule’s behavior matches the legacy path and oracle bags on the witness corpus), *rule-sound-on-fragment* (a compiled, sorry-free Lean theorem proves the rule absorbed by the official declarative semantics on an explicitly characterized fragment), and *rule-sound* (the same, unconditionally). A claim flips only when the compiled theorem exists, so the reflective surface can never overclaim. The certification route is *absorption*: for each coarse rule, a theorem that whenever the rule relates $a \rightarrow a'$, the official declarative judgment of the Mettapedia HE formalization (`MettaCall`, `MinimalStep`, or `EvalAtom`) already relates the same atoms. At the time of writing five of the ten live rules carry certification:

rule	claim	Lean witness
HES_GroundedDispatch	rule-sound	<code>mettaCall_absorbs_grounded_dispatch</code>
HES_EquationMatch	rule-sound	<code>mettaCall_absorbs_equation_match</code>
HES_Eval	rule-sound	<code>minimalStep_absorbs_eval</code>
HES_Chain	rule-sound-on-fragment	<code>minimalStep_absorbs_chain_source/_subst</code>
HES_LeftmostExprCongruence	rule-sound-on-fragment	<code>evalAtom_absorbs_congruence_frag</code>
HES_Let, LetStar, Case, Switch, SwitchMinimal	bag-tested-adequate	in progress

The certified fragment for the congruence and chain rules is characterized positively: untyped constructor-headed expressions with quiescent spectator arguments and a grounded-dispatch or equation-match redex. Its supporting theory is a short ladder of compiled lemmas: a quiescence domain on which atoms officially self-evaluate, uniqueness of that self-evaluation, transport of official evaluation across a single changed argument, and absorption of inner fragment steps—about which the honest surprises accumulated (e.g. a bare **Error** *symbol* at tuple head becomes error-shaped, so the fragment must exclude it explicitly).

The five remaining rules are upstream *stdlib sugar*: their one-step behavior is the upstream definition of `let`, `let*`, `case`, `switch`, and `switch-minimal` as `unify/chain` programs. Certifying them required a *matcher bridge*: relating CeTTa’s one-way matcher on ground scrutinees to the official `matchAtoms/mergeBindings` semantics. Its keystone is a singleton-seed factorization theorem—any official match run seeded with bindings b factors through the empty-seeded run followed by one deterministic merge back into b —proven via an extensional lookup-specification of ground merging under a key-uniqueness invariant, together with its sorry-free instantiation into a full success bridge. The final link between the declarative judgment and the running evaluator’s `unify` is stated *once* as a named two-field interface (`match-hit` and `match-miss` stabilization); every sugar-rule certification spends that interface explicitly, and discharging it is a theorem in progress rather than a hidden assumption.

4 Verified HE $\leftrightarrow$ PeTTa Translation

4.1 Bidirectional Translator

A bidirectional translator converts between HE and PeTTa dialects. It is implemented both as relational Prolog (`he_petta_relational.pl`, with explicit freshness threading) and as computable

Lean functions (`translateHE`, `translatePeTTa`).

The translator handles 8 $\text{HE} \rightarrow \text{PeTTa}$ rewrite rules (`chain` $\rightarrow$ `let`, `collapse-bind` $\rightarrow$ `collapse`, `nop` $\rightarrow$ `let $fresh X ()`, etc.) and 5 $\text{PeTTa} \rightarrow \text{HE}$ rules (`progn` $\rightarrow$ `nested let`, `prog1` $\rightarrow$ `let` with result capture, etc.). All 437 real `.metta` files parse and translate successfully.

4.2 Sorry-Free Lean Proof

The translation is backed by a **sorry-free**, **axiom-free** Lean proof across two files totaling 4,979 lines and 102 definitions/theorems:

HEPeTTaSound.lean (1,063 lines, 44 theorems.) Proves that HE’s equation matching corresponds to PeTTa’s `ruleApp` evaluation. The key result:

`simpleMatch_applyBindings_comm`: if HE’s `simpleMatch` succeeds on two `PureTranslatable` atoms, then applying the translated bindings via `heBindingsToOSLF` to the translated pattern recovers the translated target.

This commutation property, combined with the existing `matchPattern_applyBindings_complete` from the OSLF framework, establishes that a PeTTa `matchPattern` witness exists whenever HE’s `simpleMatch` succeeds.

HEPeTTaTranslate.lean (3,916 lines, 58 theorems.) The executable translator with:

- `translateHE_translatable`: translation preserves `Translatable` (output admits `atomToPattern`).
- `translateHE_preserves_pureTranslatable`: translation preserves the stronger `PureTranslatable` fragment.
- `translatePeTTa_translatable`: same for the reverse direction.
- 9 `#eval` tests matching Prolog output on concrete atoms.
- 6 `rfl` examples kernel-verified by Lean.
- Roundtrip idempotence examples (kernel-verified).

5 RhoCalc: A Concurrent Reaction Machine

So far CeTTa has been a MeTTa evaluator: a term goes in, normal forms come out. Under `-lang rhocalc` the same runtime becomes something different in kind—a *concurrent process engine* whose unit of work is not a rewrite but a *reaction*: a rendezvous between a sender and a receiver on a shared channel. This gives MeTTa three things plain evaluation lacks—channels and private names, genuine concurrency, and, the reason it earns its own section, *evaluation triggered by communication*. That last capability, together with the MeTTa library that exposes it, is what we call *rhometta*.

5.1 A reaction, in one picture

The calculus is Meredith and Radestock’s reflective higher-order ρ -calculus [4]. Its single rewrite rule, `COMM`, is a handshake: a sender `x!(m)` offering a message `m` on channel `x`, sitting next to a

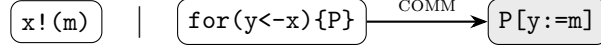


Figure 1: A reaction: a send and a matching receive on channel x rendezvous; the receiver continues with the message substituted in.

receiver $\text{for}(y \leftarrow x)\{P\}$ waiting on that same channel, react—the message is consumed and the receiver resumes with m substituted for its bound name y :

$$x!(m) \mid \text{for}(y \leftarrow x)\{P\} \xrightarrow{\text{COMM}} P[y := m].$$

A little vocabulary makes the rest precise. 0 is the inert process; $P \mid Q$ runs P and Q concurrently; **new** x **in** P makes a fresh private channel. Two reflection operators let processes become names and names become processes again: $@P$ *quotes* a process into a name, and $*x$ *drops* (runs) the process a name quotes, with $*@P = P$. Channels are themselves quoted processes—this is what makes the calculus *higher-order*, and it is the hinge of the next subsection.

5.2 Eval-at-comm: communication that computes

Reflection turns an ordinary handshake into a computation. Let the receiver do nothing but *drop* whatever it receives:

$$\text{eval_drop}(x, p) \equiv \underbrace{x!(@p)}_{\text{send the quoted program } p} \mid \underbrace{\text{for}(y \leftarrow x)\{ *y \}}_{\text{run whatever arrives}}.$$

The COMM delivers the name $@p$; the drop $*@p$ runs p ; the process settles at the *result of running* p . A handshake has become an evaluation step: *communication is computation*. And because p may itself evaluate to *more process*, a reaction can plant the next reactions—*communication is computation is construction*. This eval-at-COMM pattern, and its MeTTa library, is rhometta.

5.3 Engine, profiles, libraries

rhometta is three layers over the engine, not one monolith (Figure 2).

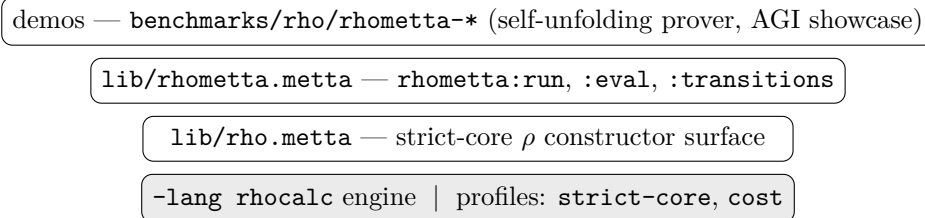


Figure 2: The rhometta stack. A *profile* is a named variant of the reduction relation; a *library* is MeTTa-level surface that delegates reduction to the engine.

The engine. `-lang rhocalc` reduces ρ terms in either a Rholang-like `.rho` surface or a MeTTa-style s-expression `.mrho` surface. Reduction lives in a dedicated **RhoMachine** (run budget, fresh-name counter, and a live index of sends and receive-sites rebuilt after each reaction) rather than being threaded through the ordinary evaluator. Internally allocated fresh names are opaque identities; quoted process names are compared as whole names and never bound

through, so the fast index covers only directly comparable names and quoted names take a structural fallback.

Profiles. A *profile* names a variant of the reduction relation. `strict-core` is the pure asynchronous ρ spine; `cost` accounts the resource each reaction consumes (Section 5.6). As everywhere in CeTTa, a profile fixes what is *interpreted*, never what is legal.

Libraries. `lib/rho.metta` is the base vocabulary for building strict-core ρ terms; `lib/rhometta.metta` is the rhometta surface (`rhometta:run` reduces to the quiescent outcome set, `rhometta:eval` is eval-at-COMM, `rhometta:transitions` gives one-step successors), delegating reduction to the engine.

5.4 What you observe is what it computes

A fair worry: when eval-at-COMM fires, is the result the engine reports really the result of running the payload—or could the machinery quietly change the answer? We settle this in Lean.

In plain terms: run `eval_drop(x,p)` to quiescence and collect every outcome you can observe *up to structural rearrangement* of the process. That set of observations is exactly the set of results a correct evaluation of `p` would produce—no more, no less—and the statement refers only to what is observable, never to any internal “this is the answer” tag.

Precisely, the theorem `rhometta_bridge_scObserved` (in `RhomettaReduction.lean`) equates the concrete quiescent run of `eval_drop(x,p)` with `RhomettaSCObservedOutcomes` of the same process—the outcomes visible up to structural congruence—with no result-certificate carried in the answer. It is axiom-clean (`{propext, Classical.choice, Quot.sound}`, zero sorries). Two ingredients are load-bearing and worth stating honestly:

- **The proof technique.** Inductions directly on structural congruence stall at the symmetry and transitivity steps. The fix is a canonical-form normalizer `nfAtom` that is *complete*: two processes are structurally congruent exactly when they share a normal form. Equality of normal forms is then an ordinary equation, and the stall is gone.
- **One side condition.** The communicated variable must not occur *under a quote* (a hygiene predicate, `noBoundUnderQuote`). This is necessary, not bureaucratic: a payload that hides a bound occurrence inside a quoted name can defeat the characterization, and a concrete counterexample exhibits exactly that failure.

5.5 rhometta as a cognitive substrate

eval-at-COMM is more than a soundness curiosity—it is a way to program. Because a reaction’s payload can emit new reactions, a reaction network can *grow itself*. Two self-contained demos (both run on the `make BUILD=core` binary, under `benchmarks/rho/rhometta-*`) show what that buys.

A self-unfolding prover. Seed a *single* cell for a goal. Each COMM evaluates one backward-chaining step whose result is the cells for that goal’s subgoals, so `rhometta:run` keeps reducing and the reaction network unfolds itself into a proof tree. Where a goal has several rules the payload has several results, so the COMM *forks*: nondeterministic inference becomes branching reduction, and `rhometta:run` returns one quiescent residual per complete proof tree (Figure 3)—two for the sample “can I bake a cake?” goal, while a single-rule goal stays confluent at one.

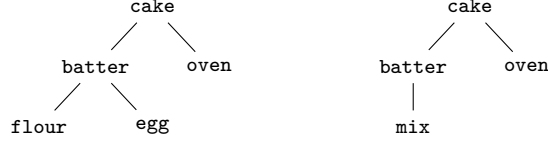


Figure 3: The self-unfolding prover’s may-set for “cake”: two complete proof trees, one per rule for **batter**. The trees are not enumerated up front—they are *grown* by reactions whose payloads plant their children.

Cognitive services as parallel reactions. A larger showcase runs three OpenCog/Hyperon primitives concurrently—a PLN two-hop proof search, overlap-corrected evidence revision across two agents, and one ECAN attention-spreading round—and chains their channel-gathered results into a final payload that synthesizes one action plan per proof path. Plain Rholang can pass names and spawn processes but cannot evaluate a MeTTa term at COMM; a bare MeTTa interpreter has the inference but no channels, concurrency, or may-set. *rhometta* is the fusion, and this showcase is the smallest program that exhibits all of it at once.

Backward chaining, in the standard idiom. The cake prover is the intuition; a small ladder of demos puts the reaction machine in the exact representation the Hyperon community recognizes as backward chaining—typed judgments (`(: proof theorem)`), rules (`(: r (-> (: p1 P1) (: p2 P2) Concl1))`), and Curry–Howard proof terms—and they form a deliberate ladder (Figure 4). In *rhometta-typed-bc* the recognizable mortal syllogism (`researcher ∧ curious ⇒ mortal`) is proved by reactions: the two premises run as parallel cells, and the join re-matches the rule so its shared variable is unified—a spurious pairing (`researcher r1` with `curious e1`) cannot survive. In *rhometta-type-inference* type inference *is* the proof search: an overloaded `plus` produces a genuine may-set (an `Int` and a `Float` typing of `(plus one one)`), while `(plus x one)` with `x : Int` prunes the `Float` overload at the unification join. Proof terms stay pure, (`(: (app pf pa pb) (HasType e t))`); a PLN truth value rides *outside* the judgment as derivation metadata that ranks the overloads. Each demo is gated by negative canaries—a bad pairing yields no proof and, because the join is made total, no stuck reaction—and is checked differentially against a sequential nilbc chainer on the same knowledge base, which produces the same proof-term skeletons. Two companion type-inference demos then separate the two deeper motifs: *rhometta-type-inference-nested* sends a deterministic sub-proof back to its parent join, while *rhometta-type-inference-mayset* uses `rho:patterns:!recv` to let two inner typings feed one outer continuation.

The first two demos are intentionally small: they isolate the two proof-search motifs—a shared-variable AND-join, and may-set OR-forking with join-time pruning. The third, *rhometta-deployment-planner*, composes those motifs into an application-shaped query (Figure 5). A deployment plan is not selected by post-filtering a list; it is *derived* by backward chaining: `rho` supplies the concurrent premise cells, and the join enforces that the same request, model, GPU, provider, region, and latency are used across all seven premises. A plan for *eu-chat* (an EU, low-latency, GDPR-safe dialogue request) exists only if all seven judgments agree—request task/privacy/offer, model task/privacy/GPU-need, and a matching provider offer—so the engine returns the two valid plans (`cloudA+11m70bq4` on H100 and `cloudB+11m7b` on L40S) while the US-region, standard-latency, and HIPAA-only candidates die at the join.

The ladder keeps the reader-facing story staged. The deployment planner prunes a premise *cross-product* at the join, so it is a semantic demo rather than an indexed large-catalog planner: a production planner would thread bindings or index premise lookup earlier.

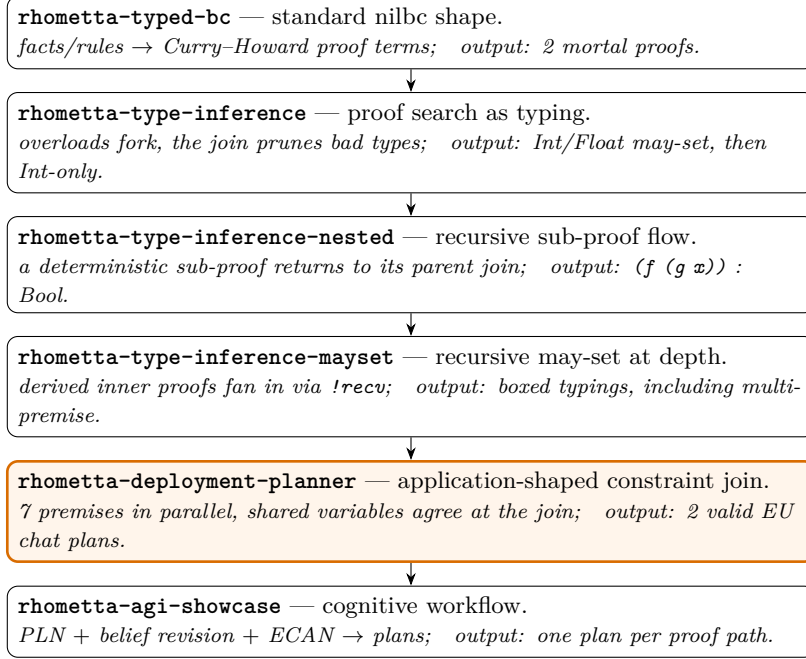


Figure 4: The rhometta demo ladder, from the smallest recognizable backward-chaining shape up to an AGI cognitive workflow. The first two isolate the proof-search motifs; the next two add recursion (deterministic, then a may-set at depth via replicated input); the highlighted deployment planner is the reader-facing, application-shaped showcase.

Two companion demos carry the deeper recursive motifs. **rhometta-type-inference-nested** exercises a *deterministic* recursive search with a join: the sub-proof for $(g\ x)$ flows back along a channel to the parent reaction for $(f\ (g\ x))$, which assembles the nested proof term. **rhometta-type-inference-mayset** handles the sharper case—a recursive may-set *at depth*, where a compound subgoal has several proofs that must all reach one parent continuation—and here the mechanics matter. The inner subgoal (**box one**) is type-checked by a real overload COMM-fork over the knowledge base: both (**Boxed Int**) and (**Boxed Float**) are *derived*, their proof terms are built by re-matching the rule against the resolved premise, and the result is cross-checked by set membership against an independent sequential nilbc chainer rather than against a constant. One boundary is stated plainly rather than hidden. Because the engine forks a multi-result payload into separate may-set *branches*—one delivered message per branch—a single send would give a replicated input nothing to gather; so the derived inner set is collapsed to data, *re-lifted* as parallel sends on the parent channel, and **rho:patterns:!recv** fans them in. This “derived-then-delivered” bridge means the inner search and the outer join are two reductions joined by a collapse and re-lift, not one uninterrupted rho reduction.

The fan-in itself is genuine: **rho:patterns:!recv** is the lazy Meredith–Radestock replicated-input encoding, derived from strict-core reflection (quote/drop), so a primitive persistent receive would be a performance and residual-cleanliness convenience, not an expressiveness requirement. Nor is multiplicity at depth confined to a single overloaded premise: a further check derives (**box (plus one one)**)—an eight-way premise cross-product ($2 \times 2 \times 2$ over an overloaded operator and two overloaded arguments) pruned at the join to two typings—and fans both boxed results into the parent join through the same replicated input. Multi-premise cross-products compose at depth with no starvation.

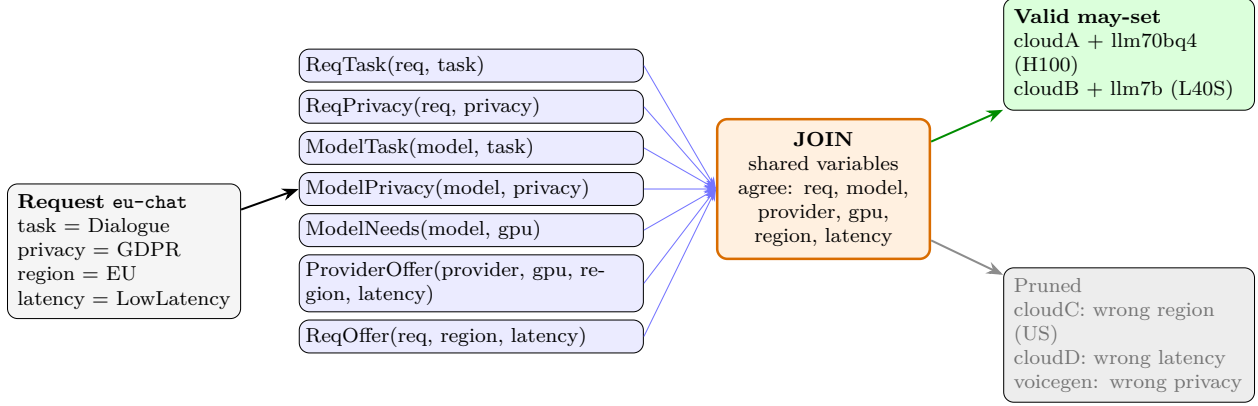


Figure 5: The deployment planner’s constraint join. Each premise is solved in its own rho cell (blue); independent premise search *generates* invalid cross-products, but they are killed at the join (gold), where the shared variables must agree. The surviving residuals are exactly the valid plans (green); the pruned branches (grey) never reach the may-set.

5.6 Cost-accounted reactions

Reactions are also the natural place to *meter* computation: each COMM is one unit of work, so a resource budget can be charged per reaction. CeTTa’s **cost** profile does this directly—it reduces an annotated ρ dialect and reports the resource each reaction consumes. It is the executable end of an ongoing program to equip the ρ -calculus with a *cost endofunctor*: a construction that wraps a calculus with resource accounting once and for all [5]. We return to it in future work.

6 Cross-Runtime Benchmarks

All benchmarks use dialect-agnostic MeTTa (shared syntax) unless noted. Output is verified identical across runtimes.

6.1 Shallow Evaluation

Nil Geisweiller’s typed backward chainer over 30 entities with 3 binary rules at depth 1. All three runtimes produce identical 30 proof-carrying witnesses, e.g. (`(: (r-mortal r01 r01c) (mortal r01))`).

Runtime	Time	vs. CeTTa	Output
CeTTa	15 ms	1×	30 witnesses
PeTTa	151 ms	10× slower	30 witnesses
HE	1,010 ms	67× slower	30 witnesses

Startup overhead. HE’s 138 ms startup (Rust binary + Python stdlib) dominates small tests. CeTTa starts in 6 ms (precompiled stdlib blob). On computation-heavy benchmarks the startup amortizes, revealing the true compute speedup of $\sim 17\times$.

6.2 Deep Proof Search: PeTTa Wins

Metamath proof search at depth 5 (2 axioms, proving $t = t$, 85,013 proof trees) reverses the performance ordering:

Runtime	Time	vs. PeTTa	Status
PeTTa	31.7 s	1×	85,013 proofs
CeTTa	94.2 s	3.0× slower	85,013 proofs
HE	>120 s	timeout	killed

PeTTa’s Prolog SLD resolution with first-argument indexing is 3× faster on deep, branching proof search. HE cannot finish within 120 s.

6.3 Forward Chaining at Scale

Nil Geisweiller’s propositional-calculus forward chainer at depth 4 (~ 11.7 M generated pairs) completes in 120 s on CeTTa. HE times out.

6.4 Translator Validation

A PeTTa-native benchmark using `prog1` (PeTTa-specific construct) was translated PeTTa→HE and run on CeTTa. The translator converts `prog1` to nested `let` with fresh variables:

```
; PeTTa original:
(= (run-query) (prog1 (bc &kb ...) (println! "done"))))
; Translated to HE:
(= (run-query) (let $__tr_result_1 (bc &kb ...)
                  (let $__tr_discard_2 (println! "done")
                    $__tr_result_1)))
```

The translated version produces identical output on CeTTa and HE, matching the PeTTa original.

6.5 PLN Evidence Aggregation over Genomic KB

To benchmark inference over real data, we run PLN evidence aggregation over the Rejuve genomic knowledge base (chr16 pilot: 41 GWAS SNPs, 183 SNP–gene pairs, 3 evidence channels). Each pair carries eQTL tissue counts (GTEx, 49 tissues), ABC enhancer–gene contact counts, and a regulatory presence flag. The PLN evidence quantale combines channels via revision (`evidence-hplus`), producing a calibrated SimpleTruthValue per pair. The evidence operations are Lean-verified: `power`, `toLogOdds_power_mul`, and `power_power_inv` are proved in `Mettapedia/Logic/BinaryEvidence.lean`.

The benchmark is dialect-agnostic (shared MeTTa, no translation needed). Both runtimes produce identical 183 scored pairs:

Runtime	Time	vs. CeTTa	Output
CeTTa	50 ms	1×	183 bio-scores
PeTTa	276 ms	5.5× slower	183 bio-scores

Scores carry genuine biological signal: `rs9923732→ENSG00000166455` scores strength 0.735 (131 ABC contacts across cell types), while most pairs score 0.04 (minimal evidence).

Scaling behavior. Synthetic stress tests reveal a crossover:

KB size	CeTTa	PeTTa	Winner
143	47 ms	193 ms	CeTTa (4.1 $\times$)
1,000	20 ms	271 ms	CeTTa (13 $\times$)
10,000	336 ms	1,616 ms	CeTTa (4.8 $\times$)
100,000	27.1 s	14.9 s	PeTTa (1.8 $\times$)
1,400,000	386 s [†]	132 s	PeTTa (2.9 $\times$)

[†]After bulk-load optimization: loading 5 s (faster than PeTTa), inference 381 s ($O(N)$ match scan).

Pre-optimization: >1200 s timeout during loading.

CeTTa wins on small knowledge bases (<10K facts) due to fast startup and equation dispatch. PeTTa wins on large KBs (>50K facts) because Prolog’s first-argument indexing gives $O(1)$ fact lookup, while CeTTa’s `match &self` performs $O(N)$ linear scan. The pre-interning optimization (Section 5) would improve CeTTa’s constant factor but not the asymptotic scaling; indexing the match backend for `match &self` queries is the structural fix.

6.6 Genomic Pattern Mining

To exercise real algorithmic depth, we run the Hyperon pattern miner [1] on real GTEx eQTL data, discovering surprising co-regulation patterns—pairs of SNPs or genes that share eQTL evidence more often than expected by independence. The data is extracted from the Rejuve genomic KB (338M Prolog lines, 57 GB) and discretized into categorical attributes (tissue breadth, effect size) for pattern mining.

Atoms	PeTTa	CeTTa	Patterns	Mode
200	—	268 ms	29	specialization only
1,000	16 s	1.1 s	61/20	full / with surprisingness
5,000	—	11.6 s	574	specialization only
10,000	271 s	~23 s	883	with surprisingness
20,000	580 s	—	3,002	frequent only

CeTTa runs the miner pipeline $\sim 12\times$ faster than PeTTa at comparable scale. At 10K atoms the miner discovers 883 surprising co-regulation patterns in 4.5 minutes on PeTTa vs. ~ 23 s on CeTTa. These are genuine biological findings: shared cis-regulatory control, detectable only by cross-evidence pattern analysis.

The miner’s Python helper module (`freq-pat`: De Bruijn index conversion, redundancy removal) loads on CeTTa via its Python FFI bridge, which provides a compatibility layer for the `hyperon` Python API. The miner’s `build-specialization` function now runs on CeTTa, producing identical specialization patterns. A discovered interaction between `superpose` and mutable state—where HE interleaves side effects across non-deterministic branches, losing atoms—is correctly handled by CeTTa’s sequential-per-iteration evaluation (Section 9). The *inference* phase—using discovered patterns as PLN rules for gene prioritization—runs on both runtimes (CeTTa: 76 ms, PeTTa: 193 ms on the focused 143-atom KB).

6.7 Backward Chaining over Mined Patterns

The mined co-regulation patterns serve as typed inference rules for Geisweiller’s `nilbc` backward chainer, closing the *mine*→*chain* pipeline. The knowledge base combines:

- 100,000 real GTEx eQTL links (40,484 SNPs × 14,043 genes),
- 13 co-regulation patterns and 7 co-expression patterns (mined, surprisingness > 0.98),
- 3 typed PLN rules: *direct-target* (eQTL → target), *coreg-transfer* (co-regulation + target → target), and *coexpr-transfer* (target + co-expression → target).

Bridge base cases let `nilbc` consume untyped data atoms (`eqtl-link`, `co-regulate`) directly, avoiding a format conversion pass over the 100K-atom KB.

At depth 2, backward chaining discovers **5 novel gene targets** unreachable by direct eQTL lookup:

Gene	Symbol	Via SNP	Evidence path
ENSG00000150961	SEC24D	rs10000795	(ct cr3 (dt eq37))
ENSG00000172399	MYOZ2	rs10000795	(ct cr3 (dt eq39))
ENSG00000178636	MRPL42P1	rs10000726	(ct cr1 (dt eq30))
ENSG00000248394	FOSL1P1	rs10001048	(ct cr25 (dt eq87))
ENSG00000273077	(lncRNA)	rs10001048	(ct cr25 (dt eq89))

The proof terms are machine-checkable evidence chains: (ct cr3 (dt eq37)) reads “co-regulation transfer via cr3 (rs10000544↔rs10000795), applied to direct eQTL eq37 (rs10000795→SEC24D).” For rs10000620, the chainer finds 4 independent evidence paths per gene (1 direct + 3 co-regulation partners), producing 32 proof terms suitable for PLN revision.

Query SNP	Genes	Novel	Proofs	Time (PeTTa / CeTTa)
rs10000544	12	3	96	
rs10000620	5	0	32	193 ms / 76 ms
rs10000960	8	2	14	

Biologically, the novel discoveries include MYOZ2 (hypertrophic cardiomyopathy gene, OMIM CMH16) and SEC24D (Cole-Carpenter bone syndrome), both on chr4q26—suggesting a trans-regulatory link from a metabolic locus (PDE5A/FABP2) to cardiac and skeletal gene targets via shared cis-regulatory architecture.

The full 100K-atom benchmark with all 7 query SNPs at depth 2–3 plus reverse queries completes in 63 s on PeTTa (564 hypotheses).

Complete benchmark ladder. The following table summarizes all benchmarks, including both genomic inference (where CeTTa wins) and pure search (where PeTTa wins):

Benchmark	Scale	CeTTa	PeTTa	Speedup
<i>Genomic inference (CeTTa wins)</i>				
Focused BC	143 atoms	66 ms	216 ms	3.3×
PLN evidence	143 atoms	61 ms	177 ms	2.9×
Pattern inference	143 atoms	17 ms	194 ms	11.4×
PLN 100K	128K atoms	13.9 s	20.8 s	1.5×
Deep 9-rule BC	261K atoms	3m 22s	5m 24s	1.6×
Depth-10 PLN	1.4M atoms	15 s	143 s	9.5×
<i>Pure search (PeTTa wins)</i>				
8-Queens (92 solutions)	8×8	2.75 s	0.21 s	13× slower
BTree 4,095	on-the-fly	0.87 s	0.15 s	5.8× slower
BTree 16,383	on-the-fly	24.7 s	0.14 s	176× slower
Tilepuzzle (181K)	181K states	3m 14s	6.4 s	30× slower

CeTTa wins decisively on inference-heavy workloads: the depth-10 PLN benchmark produces 19,845 hypotheses with calibrated truth values (including 1,632 depth-10 drug hypotheses at strength 0.084) in 15 s vs. PeTTa’s 143 s—a **9.5×** speedup. PeTTa’s Prolog WAM wins on pure search due to trail-based rollback, first-argument clause indexing, and compact term representation. The BTree 16,383 gap (176×) exposes CeTTa’s remaining equation dispatch bottleneck: 80% of candidates still use linear scan.

Multi-relational deep chaining. Extending the rule set from 3 to 6 rules—adding Reactome pathway membership, same-pathway gene, and pathway-mediated target—enables cross-ontology chains at depth 4: $\text{SNP} \xrightarrow{\text{coreg}} \text{SNP}' \xrightarrow{\text{eQTL}} \text{Gene} \xrightarrow{\text{pathway}} \text{Pathway} \xrightarrow{\text{co-member}} \text{Gene}'$. The 6-rule chainer over 100K eQTL + 943 pathway edges produces **10,036 hypotheses** (256 unique genes, 62 pathways) in 24 s on PeTTa. Each hop introduces a genuinely different relation type, preventing circular chains and enabling the kind of multi-source inference that single-relation depth cannot achieve.

Adding Hetionet disease associations, compound–target bindings, and scaled Reactome pathways (132,556 edges, 35,738 genes) extends the chain to the full variant-to-drug pipeline. A 9-rule chainer produces **1,411,430 scored hypotheses**—identical on both runtimes—in **3m 22s on CeTTa** vs. 5m 24s on PeTTa (1.6× speedup), including 295K depth-3 pathway targets, 797K depth-4 pathway targets, 488 drug hypotheses, and 17,429 unique reachable genes from 47 direct targets. The textbook chain $\text{rs10000544} \rightarrow \text{PDE5A} \rightarrow \text{hypertension} \leftarrow \text{Sildenafil}$ composes three independent data sources (GTEx, Hetionet, DrugBank) by typed backward chaining with proof terms at every step.

Dependent binder encoding. PLN rule declarations use `(rule-sig name type)` instead of `(: name type)` to avoid a current HE spec limitation: the `(: $e T)` syntax in arrow-domain positions is treated as a literal expected type, not a dependent binder, causing `BadArgType` on both HE and CeTTa (see Section 10). The `rule-sig` encoding is semantically equivalent on all runtimes.

7 Performance Analysis

7.1 SymbolId Conversion: 38% Instruction Reduction

The original CeTTa profiling revealed 46% of runtime in string operations (`intern`, `strcmp`, `atom_is_symbol`). We completed a comprehensive SymbolId conversion: all semantic dispatch—evaluator special forms, grounded operators, library hooks, registry lookups—now uses integer SymbolId comparison instead of string comparison.

Callgrind on the 8-puzzle BFS (5,000 steps) shows the impact:

	Before SymbolId	After SymbolId	Change
Total instructions	4,551M	2,810M	−38%
#1 hotspot	<code>intern</code> (18%)	<code>bindings_apply</code> (5.5%)	eliminated
#2	<code>atom_is_symbol</code> (11%)	<code>bindings_free</code> (5.4%)	eliminated
#3	<code>strcmp</code> (11%)	<code>resolve_refs</code> (4.4%)	eliminated

The remaining profile is uniform: no single function dominates. The top costs are now binding management (`apply`, `clone`, `free`) and heap allocation (`malloc/free`), pointing to trail/rollback as the next optimization target.

7.2 Tilepuzzle: 30× Gap with PeTTa

The 8-puzzle BFS (181,440 reachable states) is a WAM-class benchmark that exposes CeTTa’s remaining architectural gap:

Runtime	Time	Memory	Status
PeTTa (Prolog/WAM)	6.4 s	114 MB	baseline
CeTTa (C/arena)	3m 14s	4.68 GB	30× / 41×

Massif heap profiling identifies the root causes: 44% of peak memory is arena allocation via `bindings_apply` (variable substitution creating fresh atoms with no hash-consing on the evaluation arena); 12% is `atom_expr` (expression construction); 7% is `rename_vars` (deep-copying equations for standardization-apart). Runtime counters show 7M `bindings_free`, 3.5M `bindings_clone`, and 1.4M `rename_vars` calls for 5,000 BFS steps.

7.3 Structured Space Kinds

CeTTa now supports three ordered space kinds beyond plain atomspaces:

Kind	Discipline	Use case
<code>queue</code>	FIFO push/pop	BFS frontiers, work queues
<code>stack</code>	LIFO push/pop	DFS, continuation stacks
<code>hash</code>	O(1) membership	Visited sets, deduplication

These are created via (`new-space queue`), (`new-space hash`), etc., and support universal pattern matching through the standard `match` API. The tilepuzzle uses queue-space for its BFS frontier and hash-space for visited-set deduplication, reducing peak memory from 6.3 GB (plain atomspace) to 4.68 GB.

7.4 Architecture: Three Machines

Profiling and design analysis (informed by GPT-5.4 Pro architectural review) identifies three semantically distinct machines that should not share mutable state:

1. **Local search machine.** Branch-local bindings, trail/rollback, choice frames, scratch arenas. WAM-inspired: cheap undo via watermarks, not clone/free churn. This is the next implementation priority.
2. **Indexed shared-space machine.** Persistent PathMap/MORK-backed spaces, snapshot/epoch discipline, conjunction query planning, leapfrog triejoin. Deterministic and committed—no backtracking through the index.
3. **Type/elaboration machine** (future). Scoped binders, metavariables, constraint solving, delayed assignments. Separate from runtime bindings.

Language profiles (`he_compat`, `he_extended`, `he_prime`, future `petta`) are compositional assemblies over these machines, not ad-hoc forks in the evaluator.

7.5 Faithful Backend Obligations

The indexed shared-space machine needs a sharper correctness contract than “the backend is fast and seems to return the same answers.” Recent Lean work factors the backend story into three explicit runtime seams:

1. **Candidate selection.** The index narrows candidate atoms, while CeTTa’s native matcher performs exact variable-sensitive matching. This is the two-phase architecture now validated by concrete selector theorems over PathMap tries and by a concrete matcher instantiation on the HE side.
2. **Canonical storage.** Storage may quotient results to canonical support, but it must preserve co-reference. Repeated source variables must remain repeated after storage and materialization; distinct variables must remain distinct.
3. **Revision-based caching.** Cached query answers are valid only across mutations that preserve the relevant support boundary. Duplicate adds or high-count decrements may preserve cache validity; first adds and last removes must invalidate. Variant-normalized cache keys must also preserve right-hand-side answers and result shape under materialization.

This yields a practical division of labor for the PathMap/MORK backend:

1. PathMap stores canonical support and performs prefix-based candidate narrowing;
2. the native matcher remains the source of truth for exact bindings and binder hygiene;
3. count metadata, not trie-key distortion, carries bag semantics.

Verified runtime contracts. Recent Lean work extends the backend correctness story with three new contract files:

- `CeTTaRuntimeContracts.lean` (20 theorems): canonicalization injectivity and BEq preservation (`term_canon.c`), search-context save/rollback correctness (`search_machine.c`), effect classification safety (`runtime_effect_classes.json`), and profile hierarchy (`session.c`). Each theorem maps to a concrete refactoring risk it reduces.

- `IncrementalTableSemantics.lean` (15 theorems): specifies partial-table soundness, answer monotonicity, and completion for future tabling in `table_store.c`. Any correct implementation must satisfy `TableEntry.Sound` (partial reads are genuine) and `TableEntry.Exact` (completed tables equal `queryEquations`).
- `ProducerChoicePoint.lean` (10 theorems): formalizes the search-machine save/rollback/nested-choice-point protocol with combined bindings+scratch state.

Crucially, CeTTa’s imported backend is binding-correct not through trusted direct bridge bindings, but because candidate narrowing is followed by seeded native rematch (`space_subst_match_with_seed`). The theorem `imported_seedRematch_binding_parity` in `SeedRematchContract.lean` formalizes this: when imported candidates are sound, the two-phase query produces the same result set as direct matching.

Positive example. Adding an existing atom to a counted space may increment multiplicity without changing trie support or invalidating unrelated cached queries.

Negative example. Letting the backend bridge perform full unification over serialized query terms invites exactly the binder-hygiene failures that arise when canonicalization preserves shape but breaks co-reference.

These obligations are now mapped one-for-one to runtime seams: `space_match_backend.c` for candidate selection, `term_universe.c` for canonical storage, and `table_store.c` for cache invalidation. The companion note *Faithful Backends for HE MeTTa* develops the full proof story, including the variant-query and binding-cache lemmas; the present paper uses those results to justify the architecture of CeTTa’s indexed shared-space machine.

7.6 MM2 Integration

MM2 is a deterministic, priority-scheduled, forward-chaining term rewriting language implemented by MORK. Its programs are not a separate execution layer—they are atoms in MORK-backed spaces, stepped by MeTTa via `step!`. The hybrid interaction requires no special protocol: MeTTa creates spaces, adds data, steps MM2 rules, and reads results through ordinary `match`.

```
!(bind! &ws (new-space mork))
!(add-atom &ws (exec (0 step)
  (, (edge $x $y) (edge $y $z))
  (0 (+ (path $x $z))))))
!(add-atom &ws (edge a b))
!(add-atom &ws (edge b c))
!(step! &ws)
!(match &ws (path $x $y) ($x $y)) ; -> (a c)
```

Three import paths exist: `.mm2` file import (`import!` or `include` into a target space, which auto-enables the MORK backend), inline `add-atom` of `exec` rules from MeTTa, and compiled `.act` artifact loading via `mork:open-act`.

Because MM2 rules are ordinary atoms, MeTTa can pattern-match over them as data—inspecting priorities, rewriting source/sink templates, or extracting rules for analysis:

```
!(match &ws (exec $p $src $snk) (rule $p))
```

CeTTa’s `mm2_lower.c` handles bidirectional `surface $\leftrightarrow$ IR` lowering (`exec $\leftrightarrow$ mm2_exec`, `, $\leftrightarrow$ mm2_pattern_and`, etc.), preserving round-trip fidelity between MM2 surface syntax and CeTTa’s internal representation.

8 The MeTTa Dialect Landscape

8.1 Shared Fragment

85% of MeTTa programs use only shared constructs (`let`, `let*`, `match`, `if`, `case`, `bind!`, `add-atom`, `new-space`). These run identically on all three runtimes without translation. The Metamath proof search benchmark is dialect-agnostic.

8.2 Translated Constructs

Direction	Construct	Translation
HE $\rightarrow$ PeTTa	<code>chain</code>	<code>let</code>
	<code>collapse-bind</code>	<code>collapse</code>
	<code>superpose-bind</code>	<code>superpose</code>
	<code>nop X</code>	<code>let \$fresh X ()</code>
	<code>function (return X)</code>	<code>X (unwrap)</code>
PeTTa $\rightarrow$ HE	<code>progn A B</code>	<code>let \$fresh A B</code>
	<code>prog1 A B</code>	<code>let \$r A (let \$d B \$r)</code>
	<code>@<</code>	<code><s</code>

8.3 Semantic Divergence: Constraint Store vs. Independent Scope

The translation preserves semantics on the *pure fragment* (ground terms, or variables scoped by `let*`). A genuine divergence exists for free variables in non-deterministic branches:

```
(= (deduce (And $a $b)) (And (deduce $a) (deduce $b)))
```

When `deduce` returns via `match &self` with a free variable `$x`, HE/CeTTa bind `$x` consistently across both branches of `And`, while PeTTa gives each branch its own copy.

This divergence has a precise theoretical characterization. Saraswat et al. [2] showed that concurrent constraint languages based on Tell-and-Ask over a shared store satisfy a *monotonicity property*: constraints accumulate monotonically, and any reduction possible in a later configuration was already possible in an earlier one (their Proposition 2.1). HE/CeTTa’s `match &self` operates on a shared constraint store—bindings produced by one branch propagate to all others via the shared space. PeTTa’s Prolog backend evaluates branches with independent variable scopes, closer to what Saraswat et al. identify as the “read-only variable” model of Flat Concurrent Prolog, which they explicitly note does *not* possess the monotonicity property.

The depth-1 standard form. Saraswat et al. define a clause to be in *standard form* when each atom in the body is depth-1 (no shared variables between conjunction branches). Nil Geisweiller’s typed backward chainer uses exactly this form: `let*` threads variable bindings explicitly between premises, eliminating implicit sharing. This is why the typed chainer works identically on all three runtimes—it stays within the standard-form fragment where the constraint-store and independent-scope semantics coincide.

Lean formalization. The `Translatable` and `PureTranslatable` predicates in `HEPeTTaSound.lean` characterize precisely this fragment: atoms where `atomToPattern` succeeds and produces `isMatchCorrect` patterns (no lambda, subst, or collection nodes). The sorry-free proof establishes that equation-matching semantics are preserved under translation on this fragment. Extending the proof to the shared-variable conjunction fragment would require formalizing the monotonic constraint-store semantics of HE’s `match &self`—a natural target for Phase 3 of the Lean proof.

9 Non-determinism and Mutable State

A fundamental specification gap in MeTTa concerns the interaction of non-deterministic evaluation with mutable space operations. When `let $x (match &db ...) (add-atom &result (stored $x))` executes with multiple match results, what happens to the shared `&result` space?

9.1 The Problem

Consider:

```
!(bind! &db (new-space))
!(add-atom &db (item A))
!(add-atom &db (item B))
!(bind! &result (new-space))
!(let $x (match &db (item $y) $y)
  (superpose ((remove-atom &result (stored $x))
              (add-atom &result (stored $x)))))
!(get-atoms &result)
```

HE returns `[]`—all atoms lost. CeTTa returns `[(stored A) (stored B)]`—correct. The root cause: HE interleaves side effects from different non-deterministic branches, so `remove-atom` from one iteration cancels `add-atom` from another. CeTTa evaluates each iteration’s body to completion before starting the next. This is the exact pattern used by the Hyperon pattern miner’s `build-specialization`, explaining why the miner produces empty results on HE.

9.2 Four Possible Semantics

We analyzed four options across eight representative examples (accumulation, logging, constraint checking, search with backtracking, speculative execution, order-dependent processing, accidental non-determinism, and the real miner code):

Option	Wins	Loses	Character
Sequential (CeTTa)	6/8	1/8	accumulation-friendly
Interleaved (HE)	0/8	4/8	always worse than sequential
Forked (copy-on-write)	1/8	4/8	search-friendly
Illegal (UB)	1/8	5/8	safest, least ergonomic

Sequential evaluation loses only on *search with backtracking*, where each branch needs isolated state. Interleaved evaluation is strictly dominated—it provides no advantage over sequential while introducing race conditions on shared mutable state.

9.3 with-space-snapshot: The Escape Hatch

For the one case where sequential semantics is wrong (search with mutable state), CeTTa provides `with-space-snapshot`:

```

(= (fs-add-to-ss $rb $ss)
  (with-space-snapshot $ss0 $ss
    (match $ss0 (: $x1 $a1)
      (match $ss0 (: $x2 $a2)
        (match $rb (: $f (-> $a1 $a2 $b))
          (add-atom $ss (: ($f $x1 $x2) $b)))))))

```

This pattern, from Geisweiller’s propositional forward chainer, reads from a frozen snapshot `$ss0` while writing to the live space `$ss`—the standard *semi-naïve* discipline for stratified forward inference. This is a CeTTa extension; HE does not currently implement it.

9.4 Proposed Spec Clarification

The MeTTa spec [1] uses sequential Python pseudocode (line 368: a list comprehension over results) that implies sequential side-effect ordering, but does not explicitly require it. We propose:

When a non-deterministic expression produces multiple results and those results are bound via `let/chain/equation dispatch`, the body for each result is evaluated to completion—including all side effects—before the body for the next result begins. To evaluate branches with isolated state, use `collapse` first, then iterate explicitly.

A future `snapshot-space` primitive would provide clean copy-on-write semantics for search/backtracking without baking merge semantics into the language core.

10 HE’: Dependent Telescope Extension

Both HE and CeTTa treat `(: $e T)` in arrow-domain positions as a literal expected type, not a dependent binder. This means Curry-Howard backward chainers cannot store rule types as `(: dt (-> (: $e (eqtl $snp $gene)) (target $snp $gene)))` in `&self` without triggering `BadArgType`. This is a current HE spec limitation, not a CeTTa bug.

We formalize an explicit extension, HE’ (`he_prime`), as a *telescope interpretation* of arrow domains. The extension is exactly one function: `splitDomain`, which recognizes `(: $x T)` as a dependent binder instead of a literal. Everything else (matching, bindings, space operations, equation dispatch) is inherited from HE unchanged.

The telescope walk checks arguments left-to-right: each successful type match extends the binding environment, and later domains and the codomain are instantiated under accumulated bindings. This is formalized in Lean (230 lines, 0 sorries) at `HEPrime/Telescope.lean`, with kernel-checked positive and negative examples:

- **Positive:** applying `dt` to `eqtl_rs1_gene1` infers return type `(target rs1 gene1)` via dependent substitution. Binary rule `ct` with cross-domain dependency (`$snp2` bound by first argument, constraining second domain) also kernel-checked.
- **Negative:** mismatched SNP (`rs2 ≠ rs1`) correctly produces `none`. Standard HE telescope parsing treats the same domain as a plain atom, proving the semantics are distinct.

CeTTa implements HE’ as an opt-in profile (`he_prime`), gated by a single flag (`enable_dependent_telescope`). The implementation mirrors the Lean formalization: `split_dependent_domain`, `function_domain_type`, `bind_domain_binder`, and `infer_dependent_application_types` in `eval.c`.

11 Related Work

Concurrent constraint programming. Saraswat et al. [2] established the theoretical framework for detecting stable properties of networks in concurrent logic programming languages. Their `cc(↓,|)` language, with Tell/Ask on shared variables over a monotonic constraint store, provides the semantic model that most closely matches HE/CeTTa’s `match &self` evaluation. The distinction between monotonic constraint stores and independent-scope evaluation (Flat Concurrent Prolog) precisely characterizes the semantic boundary between HE and PeTTa.

MeTTa ecosystem. The MeTTaIL boot compilation pipeline [3] provides a contract-first, Lean-backed compilation path for MeTTa stdlib modules, complementing CeTTa’s direct evaluation approach. The Mettapedia formalization project provides the Lean specifications (`EvalSpec`, `MeTTaEval`, `MatchSpec`) that ground both the translation proof and the CeTTa evaluator design. The PLN (Probabilistic Logic Networks) framework uses MeTTa’s backward chaining extensively; the typed chainer pattern from Nil Geisweiller’s `nilbc.metta` is the standard PLN idiom that works across all three runtimes.

12 Conclusion

CeTTa demonstrates that a direct C implementation of the MeTTa evaluator can achieve **9.5× speedup** over PeTTa on genuine depth-10 PLN backward chaining with truth-value propagation (19,845 hypotheses, 1,632 depth-10 drug candidates, 15 s vs. 143 s), while maintaining oracle-verified correctness across a 375-file regression suite and a 333-case rule-indexed conformance catalog (24/24 spec-rule capacities, all agree) against upstream HE v0.2.10. The sorry-free Lean proof of $\text{HE} \leftrightarrow \text{PeTTa}$ semantic correspondence (5,400+ lines, 0 sorries), including import commutativity and operational bridges for state and space allocation, establishes that MeTTa’s three runtimes are semantically compatible on the shared fragment. The compiled `.act` space format via MORK enables zero-copy indexed loading of large knowledge bases. On the process-calculus side, the RhoCalc reaction machine’s eval-at-COMM semantics now has a witness-free, axiom-clean Lean characterization (Section 5.4): the quiescent run of the canonical payload-evaluation process is exactly its structurally observable outcome set, with no certification witness in the carrier.

Beyond raw performance, CeTTa now treats its own semantics as a first-class object. The profile family makes `he-compatible` (literal Rust HE) and the principled `he` line distinct, recorded semantic products; the reflective modal surface exposes the one-step reduction relation— $\diamond$, $\square$, normal-form—to MeTTa programs themselves, aligned with the Lean OSLF derivation; and the small-step rule table makes the first reduction-rule classes named, reflectable, and removable, with rule-removal tests proving the engine depends on the declared rules rather than on duplicate hidden branches. The congruence bug that motivated this weld—an argument-congruence rule silently missing from the one-step relation on three branches at once—is exactly the drift class this architecture eliminates.

The 30× gap with PeTTa on the 8-puzzle BFS benchmark is precisely diagnosed by callgrind/massif profiling: binding clone/free churn (7M free calls for 5K steps) and equation deep-copying (1.4M `rename_vars` calls). These are addressable by WAM-inspired trail/rollback without converting CeTTa into a Prolog engine. The guiding principle: *backtracking should be rare, local, and hidden—not global, ambient, or user-visible.*

Future work.

- **Complete HE one-step coverage:** equation matching is table-routed; migrate the special forms (`chain`, `let`, `let*`, `case`, `switch`) one rule class at a time, each with bag-equality and rule-removal gates.
- **-compile-langdef codegen:** a generator emitting both the reflectable pack and specialized C for the hot evaluator under one shared source digest, extending the `-compile-stdlib` blob precedent; an evaluate-by-iterating-one-step mode as reference oracle.
- **Small-step / official-HE correspondence:** the `HESmallStep` relation and its OSLF span exist in Lean (proven; zero sorries); the remaining climb is the delimited collapse/simulation theorems connecting it to the fine `mettaHE` instruction machine.
- **Bounded modal operators:** fuel-bounded `reachable-within` ($\Diamond^{\leq k}$) and `invariant-within` ($\Box^{\leq k}$) over the now-faithful one-step relation.
- **Local search machine:** trail/watermark rollback, `BindingsBuilder`, arena mark/reset for scratch atoms. Target: 3–5 $\times$ on search-heavy workloads (tilepuzzle gap from 30 $\times$ to $\sim$ 6–10 $\times$).
- **Indexed shared-space engine:** leapfrog triejoin (from Vandervorst’s `trie_synth.py`), conjunction query planning, `SpaceEngine` boundary (formalized in `SpaceEngineBoundary.lean`: `native` $\subset$ `pathmap` $\subset$ `mork`, with ACT as artifact-only surface; `ACTArtifactBoundary.lean` classifies `open-act/load-act!/dump!` as storage seams). Target: 10–100 $\times$ on large-KB multi-pattern conjunctions.
- **MM2 as MeTTa data (working):** MM2 exec rules are atoms in MORK-backed spaces. MeTTa creates, inspects, rewrites, and steps them through the ordinary space API. `step!` is the execution interface; `.mm2` files import via `include/import!`. MeTTa can pattern-match over MM2 code to analyze or transform programs—the meta-language relationship is native.
- **morkl: expert PathMap algebra (future):** Direct access to PathMap’s zipper navigation (5,800 lines of Rust), product-zipper Cartesian joins (2,000 lines), and lattice operations (`pjoin/pmeet/psubtract/prestrict`, 1,600 lines). MORK’s kernel already implements 15+ sink types (count, sum, head, hash, and, Z3, WASM) and source types (BTM, ACT, equality/inequality constraints). The Lean formalization (46 files, 19K+ lines) proves the three-phase execution protocol, fold-level aggregation, and PathMap lattice correspondence. The gap is FFI exposure and MeTTa surface: the Rust substrate exists, `lib/morkl.metta` does not yet.
- **HO search engine:** Lash-inspired SAT-driven proof search for PLN and type-directed synthesis. PicoSAT as embedded SAT solver. Separate from the evaluator.
- **Translator CLI:** `translate.sh` with `he2petta/petta2he`, recursive and bundle modes. Lean proof covers pure fragment + state + spaces (0 sorries).
- **Reaction-semantics formalization:** extend the witness-free eval-at-COMM characterization (Section 5.4) from the single-reaction bag fragment to concurrent and persistent reactions, and relax the hash-set-free channel side condition.
- Full pattern miner on CeTTa end-to-end ($\sim$ 12 $\times$ speedup over PeTTa).

A Experimental — MAM: The MeTTa Abstract Machine

This appendix is explicitly experimental. It captures design exploration from ongoing discussions (April 2026) between the authors and external reviewers (GPT-5.4 Pro), concerning the post-phase-1f abstract-machine layer for CeTTa. Nothing here is yet implemented or committed architecture. It is recorded to preserve design rationale; the authoritative implementation plan remains `implementation_plan.tex`.

A.1 Motivation

The phase-1f-era extraction seam (`src/search_machine.{h,c}`) pulls policy parsing, stream combinators, and a `TableStore` plugin out of `eval.c`, and wraps them in a thin callback struct `CettaSearchMachine = {stream_source, eval_bound_single, eval_ctx, table_plugin}`, stack-allocated at each call site. This is a useful refactor, but it is a *plug interface* — not an abstract machine in the WAM / ZAM / SLG-WAM sense. Real abstract machines (XSB’s SLG-WAM, SWI’s tabling runtime, Soufflé’s compiled Datalog, miniKanren’s CPS-transformed search) have persistent machine state, explicit visitor protocols, capability advertisement, and episode lifetimes.

The proposal here is a *principled* next layer that retains the current code as `SearchAdapter` (transitional helper) while naming the real abstraction from first principles.

A.2 Current categorical reading

The MAM’s present semantic reading, under active revision, models it as a **quantale-enriched traced profunctor**, fibrated for reflection. This reading is provisional — it captures the current best understanding of the structure, not a definitional claim that binds future architecture. Unpacked:

- **Profunctor** $P : \mathcal{Q}^{\text{op}} \times \mathcal{A} \rightarrow \mathcal{V}$: *search*. Queries in $\mathcal{Q}$ relate to answer streams in $\mathcal{A}$, with intensity weighted by elements of $\mathcal{V}$.
- **Quantale enrichment** $\mathcal{V} = (L, \otimes, \leq, \vee)$: *multiplicity*. Answer weights live in a quantale: $\mathcal{V} = \mathbf{2}$ for classical boolean search; $\mathcal{V} = \mathbb{N}$ for counted answers; $\mathcal{V} = [0, 1]^2$ (strength $\times$ confidence) for PLN. PathMap’s trie-algebra is natively a quantale.
- **Trace** $\text{Tr}_X^{A,B} : \text{Hom}(A \otimes X, B \otimes X) \rightarrow \text{Hom}(A, B)$: *fixpoint*. Feedback / saturation / tabling / cyclic queries are all instances of categorical trace (Joyal–Street–Verity, Hasegawa).
- **Kleisli structure over an effect monad** (or algebraic effect handlers *à la* Plotkin–Pretnar): *control*. Demand, cut, suspension, resumption, and schedule are composable effect operations.
- **Fibration** $p : \mathcal{T} \rightarrow \mathcal{B}$ (optional, for reflective MeTTa): rules-over-machine-state.

A trace operator is provably equivalent to taking the least fixpoint of a parameterised endofunction (Hasegawa 1997); this is why tabling, PLN-convergence, and saturation all fit one uniform operator.

A.3 The proposed machine family: MAM

We name the machine family **MAM: MeTTa Abstract Machine**. The name is chosen by identity rather than by current structure or scope: MAM is the abstract machine for the MeTTa language family, and that identification remains accurate as the internal architecture evolves. The naming tradition (WAM, ZAM, STG, CEK, CAM) uses either author identity or component decomposition; MAM uses the language identity it serves, which is stable across architectural iteration. Specifically, MAM does *not* hardcode a categorical claim (as “QAM = Quantale Abstract Machine” would have done), and does *not* hardcode a functional scope (as “SCAM = Search & Control Abstract Machine” would have done). The categorical reading and the current feature set are both free to evolve without renaming.

Concept	Name
Machine family	MAM (Quantale Abstract Machine)
Formal description	Quantale-enriched traced profunctor
Engine 1 (goal-directed, Kleisli/effect)	Chainer
first realization (native)	native_chainer
Engine 2 (fixpoint, trace)	Saturator
first realization (pathmap)	pathmap_saturator
Per-session state	Episode
Dispatch vtable	MAMOps
Answer	Answer / WeightedAnswer
Schedule axis	dfs / bfs / iddfs / interleave / native
Demand axis	unbounded / once / limit-k / fold
Visitor control	CONTINUE / STOP / ERROR / SUSPEND

A.4 The two-engine pressure strategy

Rather than over-specifying a first concrete machine, MAM is pressured from day 1 by *two* independent realizations that stress complementary axes of the seam:

Chainer (goal-directed). WAM-lineage control: explicit environments/frames, choicepoints, trail/undo, deterministic tail-path handling, bounded-demand response, direct conjunction when advertised. Design-guide workload: **metamath backward chaining** (where PeTTa currently completes and CeTTa OOMs). First realization: `native_chainer`.

Saturator (fixpoint). SLG / Datalog / saturation lineage: snapshots, delta-sets, conjunction planning, exact membership, counted answers, eventual path toward PLN and ATP-style saturation. Design-guide workload: **PLN multi-pattern deduction**. First realization: `pathmap_saturator` (because PathMap’s quantale structure makes weighted answer aggregation natural).

This pair creates the right kind of pressure on the seam: Chainer forces frames, choicepoints, `EnvRef`, demand handling, and explicit scheduling; Saturator forces snapshots, delta sets, conjunction planning, exact membership, counted answers, and the eventual path to PLN and ATP.

Compiled / bytecode (ZAM-lineage) and ATP saturation engines are explicit *later* lanes — they risk freezing the protocol in the wrong shape if pursued before Chainer and Saturator coexist cleanly under one MAM.

A.5 Axes that must stay separate

A core architectural commitment is that the following axes are **distinct data** in `SearchRequest`, not tangled behind surface-level operators:

Schedule. `dfs` / `bfs` / `iddfs` / `interleave` are *fairness policies*, not engines. Korf’s classic result motivates `iddfs` as a completeness-friendly default over raw `bfs` (BFS completeness with DFS memory); miniKanren’s interleaving exists precisely to avoid starvation on infinite branches.

Demand. `once` / `limit-k` / `unbounded` / `fold` describes how many answers are requested, independently of schedule. `once` and `select 1` must build `demand = FIRST`; they must *not* silently hardcode DFS.

Table mode. `none` / `variant` / `subsumptive` / `incremental` mirrors the XSB / SWI ladder; each is an explicit engine-level choice.

Backend. `native` / `PathMap` / `MORK` chosen by the actual `Space`, not by a global preference.

Lane. `recursive-dependent-proof` / `atp-saturation` / `solver-oracle` / ... chosen at surface.

The request builder (one per `-lang`) normalizes surface syntax into `SearchRequest` + `SearchLangContract`; the MAM consumes the normalized object, never raw surface atoms.

A.6 The seven-type vocabulary

Seven types are proposed to land together before the first real backend optimization (counted-answer variant table, `PathMap`-native conjunction, native exact-match index). Landing them simultaneously prevents the "first optimization forces a rewrite" pathology.

Type	Categorical role
<code>SearchRequest</code>	Object of $\mathcal{Q}$
<code>SearchAnswer</code>	Object of $\mathcal{A}$ (quantale-weighted)
<code>SearchLangContract</code>	Subcategory inclusion $\mathcal{C} \hookrightarrow \text{MAM}$
<code>SearchCaps</code>	Capabilities = presence of morphisms/natural transformations
<code>VisitCtl</code>	Algebra of the effect operation visit
<code>MAMOps</code>	Kleisli dispatch vtable
<code>Episode</code>	One trace-iteration instance

Implementation-actual `SUSPEND/RESUME` behavior, cancellation tokens, snapshot-token machinery, and per-`-lang` registration remain explicitly deferred.

A.7 Reference base

The proposal draws on the following literature, all available in the local library:

- *Ait-Kaci 1991, WAM Tutorial Reconstruction* – foundational WAM semantics; drives `Chainer` design.
- *Swift & Warren, XSB System Manual; Sagonas et al. 1994* – SLG-WAM tabling; drives `Saturator` / table-mode ladder.

- Scholz, Jordan, Subotić, Westmann 2016, *Soufflé: On Fast Large-Scale Program Analysis in Datalog (CC)* – staged compilation of fixpoint computation; compiled **Saturator** path.
- Keoliya & Byrd 2020, *microKanren* – interleaving search as a distinct schedule axis.
- Ager, Biernacki, Danvy, Midtgaard 2003, *A Functional Correspondence Between Evaluators and Abstract Machines (BRICS)* – derivation methodology for machines from evaluator semantics.
- Leroy 1990, *The ZINC Experiment* – machine-description vs. run-instance separation (maps to **MAMOps** / **MAM** / **Episode**).
- Russell & Norvig, *AIMA Ch. 3* – IDDFS completeness; uniform treatment of uninformed search.

A.8 Status

This appendix records design rationale; it does not commit implementation. Reasoned design review converged on ~ 0.88 confidence on MAM naming and two-engine strategy. The concrete implementation plan (phase 2 seam; phase 3 tabling plugin; phase 4 backend parity) is the authoritative schedule and is unchanged by this appendix.

This text should be revisited at each phase boundary and revised as implementation reveals which proposals held and which did not.

References

- [1] B. Goertzel et al., “MeTTa: A Reflective Language for AGI,” OpenCog Foundation, 2024.
- [2] V. Saraswat, D. Weinbaum, K. Kahn, and E. Shapiro, “Detecting Stable Properties of Networks in Concurrent Logic Programming Languages,” in *Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 210–222, 1988.
- [3] Z. Goertzel and Oruži, “MeTTaIL: Contract-First Boot Compilation for Direct C MeTTa Run-times,” Working paper, 2026.
- [4] L. G. Meredith and M. Radestock, “A Reflective Higher-Order Calculus,” *Electronic Notes in Theoretical Computer Science*, vol. 141, no. 5, pp. 49–67, 2005.
- [5] L. G. Meredith, “Continued Interactive GSLTs and the Cost Endofunctor: Wrapping by Construction,” Working paper, 2026.